

Problemes 2

- 2.1. **(Revisió)** Un professor rep n sol·licituds de revisions d'examen. Abans de començar, el professor mira la llista dels n estudiants que han sol·licitat revisió i pot calcular, per a cada estudiant i , el temps t_i que utilitzarà per atendre l' i -èsim estudiant. Per a estudiant i , el temps d'espera e_i és el temps que el professor triga a revisar els exàmens dels estudiants que fan la revisió abans que i .

Dissenyeu un algorisme per a computar l'ordre en que s'han de revisar els exàmens dels n estudiants de manera que es minimitzi el temps total d'espera: $T = \sum_{i=1}^n e_i$.

- 2.2. **(Subgraf induït)** Tenim un graf no dirigit $G = (V, E)$. Donat un subconjunt $V' \subseteq V$ el *subgraf induït* per V' és el graf $G[V'] = (V', E')$ on $E' = E \cap (V' \times V')$, és a dir, conté totes les arestes que tenen els dos extrems a V' . El *grau* d'un vèrtex a un graf és el nombre d'arestes incidents al vèrtex. Doneu un algorisme eficient per al següent problema: donat G i un enter positiu k , trobar el subconjunt (si hi ha algun) més gran V' de V , tal que cada vèrtex a V' té grau $\geq k$ a $G[V']$.

- 2.3. **(Partició interval)** El problema de la *partició interval* (*Interval Partitioning Problem*) és similar al problema de la selecció d'activitats vist a classe però, en lloc de tenir un únic recurs, tenim molts recursos (és a dir, diverses còpies del mateix recurs). Doneu un algorisme que permeti programar totes les activitats fent servir el menor número possible de recursos.

- 2.4. **(Recobriment minimal)** Sigui $G = (V, E)$ un graf no dirigit. Un subconjunt $C \subseteq V$ s'anomena *recobriment de vèrtexs* de G si

$$\forall (u, v) \in E : \{u, v\} \cap C \neq \emptyset.$$

Donat $G = (V, E)$, un recobriment de vèrtexs C és diu que és *minimal* si, per qualsevol $C' \subseteq V$ tal que $C' \subset C$, hem de tenir que C' no és un recobriment de vèrtexs.

Un conjunt $C \subseteq V$ és un *recobriment de vèrtexs mínim* si C és un recobriment de vèrtexs a G amb mínima cardinalitat. (Quan G és un arbre, hi ha un algorisme polinòmic per a trobar un recobriment de vèrtexs mínim a G , però per a G generals el problema és NP-hard).

En aquest problema demostrareu que, a diferència del problema de trobar un recobriment de vèrtexs mínim, el problema de trobar un recobriment de vèrtexs minimal pot ser resolt en temps polinòmic.

- (a) Demostreu que un recobriment de vèrtexs minimal no necessàriament ha de ser un recobriment de vèrtexs mínim.
 - (b) Demostreu que tot recobriment de vèrtexs mínim també és minimal.
 - (c) Doneu un algorisme polinòmic per trobar un recobriment de vèrtexs minimal a G .
- 2.5. **(Ratolins i caus)** A un experiment sociològic es disposen n ratolins a les posicions $R = \{r_1, \dots, r_n\}$ determinades sobre una línia recta. A la mateixa línia hi ha n caus localitzats a les posicions $C = \{c_1, \dots, c_n\}$. A cada cau només es podrà allotjar un ratolí.

Un ratolí pot romandre a la seva posició, moure's un pas cap a la dreta de la línia (és a dir, de la posició x a la posició $x + 1$), o moure's un pas cap a l'esquerra de la línia (és a dir, de la posició x a la posició $x - 1$). Qualsevol d'aquests moviments consumeix un minut de temps.

Dissenyeu un algorisme, el més eficient possible, per a assignar els ratolins als caus de manera que es minimitzi el temps en què l'últim ratolí entra dins d'un cau. Raoneu el cost de l'algorisme proposat.

- 2.6. **(SOS Rural)** Considerad un camino rural en el Pirineo con casas muy dispersas a lo largo de él. Por motivos de seguridad se quieren colocar estaciones SOS en algunos puntos de la carretera. Los expertos indican que, teniendo en cuenta las condiciones climáticas de la zona, se debería garantizar que cada casa se encuentre como máximo a una distancia de 15km, siguiendo la carretera, de una de las estaciones SOS.

Proporcionad un algoritmo eficiente que consiga este objetivo utilizando el mínimo número de estaciones SOS posibles. Justificad la corrección y el coste de vuestro algoritmo e indicad la complejidad en tiempo de la solución propuesta.

- 2.7. **(Graf centre)** Sigui $T = (V, E)$ un arbre no dirigit amb n vèrtexs. Per a $u, v \in V$, sigui $d(u, v)$ el nombre d'arestes del camí de u a v en T . Definim el *centre* de T com el vèrtex

$$c = \operatorname{argmin}_{v \in V} \{ \max_{u \in V} d(u, v) \}.$$

Describiu un algorisme de cost $O(n)$ per a obtenir un centre de T .

- 2.8. **(Huffmann màxim)** Si utilitzem l'algorisme de Huffmann per a comprimir un text format per n , símbols que apareixen amb freqüències $f_1, f_2, f_3, \dots, f_n$ quina és la màxima longitud de compressió d'un símbol que podem obtenir? Doneu un exemple de les freqüències on es compleix aquesta condició.

- 2.9. **(Googol)** La companyia Googol vol detectar a la web en quin idioma està escrita cada pàgina a la web. Per això, ha dissenyat un programa que quan descarrega una pagina, pot demanar si la pàgina està escrita en idioma L_i ($1 \leq i \leq 8$) i el programa respon SI o NO. Googol sap que de totes les pàgines a internet, el 40% estan escrites en L_1 , el 17% en L_2 , el 15% en L_3 , el 11% en L_4 , el 9% L_5 , el 5% L_6 , el 2% L_7 , i el 1% L_8 .

- (a) Googol, ha dissenyat un programa al que, quan descarrega una pagina, pot demanar si la pàgina esta escrita en idioma L_i ($1 \leq i \leq 8$) i el programa respon SI o NO. En quin ordre s'han a de fer les preguntes per que el nombre mitjà de preguntes necessaries per a identificar l'idioma sigui mínim?
- (b) Un dels informàtics a Googol pensa que potser seria millor fer servir un programa que, donats dos conjunts disjunts d'idiomes, diu si la pàgina està escrita amb un idioma del primer conjunt o amb un idioma del segon. Dissenya un algorisme, que fent servir aquest tipus de qüestions, trobi l'idioma en el que està escrit la pàgina. Quin serà el nombre esperat de qüestions utilitzant el teu algorisme? (Ajut: Penseu en el codis de Huffman)
- 2.10. **(Bottleneck ST)** Un *bottleneck spanning tree* T d'un graf no dirigit i ponderat $G = (V, E, w)$, on $w : E \rightarrow \mathbb{R}^+$, és un arbre d'expansió de G on el pes més gran és mínim sobre tots els arbres d'expansió de G . Diem que el valor d'un bottleneck spanning tree és el pes de la aresta de pes màxim a T .

- (a) Demostreu la correctesa o trobeu un contraexemple pels enunciats següents:
- Un *bottleneck spanning tree* és també un arbre d'expansió mínim.
 - Un arbre d'expansió mínim és també un *bottleneck spanning tree*.
- (b) Doneu un algorisme amb cost $O(|V| + |E|)$ que donat un graf G i un enter b , determini si el valor d'un bottleneck spanning tree és $\leq b$.
- 2.11. **(MST?)** Aquí teniu el pseudocodi de tres algorismes diferents. Tots tenen com a entrada un graf connex ponderat $G = (V, E, w)$, amb pesos no negatius a les arestes, i retornen un subconjunt d'arestes F .

Per cadascun dels algorismes, heu de determinar si (V, F) és un arbre d'expansió amb cost mínim o no, i justificar la vostra resposta amb la demostració adient. A més, heu de donar la implementació més eficient de cadascun dels algorismes, independentment de si retornen o no un MST.

(a) MAYBE-MST-A(G, w)

```
sort the edges into nonincreasing order of edge weights w
T = E
for each edge e, taken in nonincreasing order by weight
    if T - {e} is a connected graph
        T = T - {e}
return T
```

(b) MAYBE-MST-B(G, w)

```
T = ∅
for each edge e, taken in arbitrary order
    if T + {e} has no cycles
        T = T + {e}
return T
```

(c) MAYBE-MST-C(G, w)

```
T = ∅
for each edge e, taken in arbitrary order
    T = T + {e}
    if T has a cycle c
        let e' be a maximum-weight edge on c
        T = T - {e'}
return T
```

2.12. **(Mark and collect)** Tenim un graf connex no dirigit $G = (V, E)$ on cada node $v \in V$ té associat un valor $\ell(v) \geq 0$; considerem el següent joc unipersonal:

- (a) Els nodes inicialment no estan marcats i la puntuació del jugador és 0.
- (b) El jugador selecciona un node $u \in V$ no marcat. Sigui $M(u)$ el conjunt de veïns de u a G que ja han sigut marcats. Aleshores, s'afegeix a la puntuació del jugador el valor $\sum_{v \in M(u)} \ell(v)$ i marquem u .
- (c) El joc es repeteix fins que tots els nodes siguin marcats o el jugador decideixi finalitzar la partida, deixant possiblement alguns nodes sense marcar.

Per exemple, suposeu que el graf té tres nodes A, B, C on A està connectat a B i B amb C , amb $\ell(A) = 3$, $\ell(B) = 2$, $\ell(C) = 3$. En aquest cas, una estratègia òptima seria marcar primer A , després C i finalment B . Aquest ordre dona al jugador una puntuació total de 6.

- (a) És possible obtenir una puntuació millor deixant algun dels nodes sense marcar? Justifiqueu la vostra resposta.
- (b) Dissenyeu un algorisme voraç per tal d'obtenir la millor puntuació possible. Justifiqueu la seva correctesa i doneu-ne el cost.
- (c) Suposeu ara que $\ell(v)$ pugui ser negatiu. Continua el vostre algorisme proporcionant la puntuació màxima possible?
- (d) Considereu la següent modificació del joc per aquest cas en què els nodes poguessin tenir valors negatius: eliminem primerament de G tots els nodes $v \in V$ amb $\ell(v) < 0$, per tal de seguidament executar el vostre algorisme sobre el graf resultant d'aquesta eliminació. Doneu un exemple on aquesta variació no proporcionï la màxima puntuació possible.

- 2.13. **(Gotes)** Considereu una canonada d'alçada L . La canonada té n gotes d'aigua a n alçades diferents dins d'ella. Les posicions on es troben les gotes es representen per nombres enters dins del rang $0 \dots L$. La posició 0 és la base de la canonada (el final) i la posició L n'és la part més alta.

Cada gota d'aigua es mou cap al final de la canonada a velocitat uniforme. Les gotes es mouen a diferents velocitats. La velocitat d'una gota és un nombre racional. Quan una gota d'aigua atrapa a una altra gota es barregen, i la nova gota assumeix la velocitat de la gota d'aigua atrapada (és a dir, la velocitat de la gota que estava més propera a la base de la canonada). Doneu un algorisme, el més eficient que pugueu, per determinar el nombre de gotes que surten de l'extrem final de la canonada. Justifica la seva correctesa, i dona el seu cost en funció d' n .

- 2.14. **(CinemaVis)** L'empresa CinemaVis ha de programar l'aparició d'un seguit d'anuncis a una pantalla gegant a la Plaça del Mig la diada de Sant Jordi. La tirada d'anuncis es pot iniciar a temps 0 (l'inici programat) però mai abans. A més, CinemaVis disposa d'un conjunt de n anuncis per fer la selecció. L'anunci i té una durada de 1 minut i té associat dos valors reals no negatius t_i i b_i . L'anunci pagarà b_i euros a CinemaVis si l'anunci i s'emet a l'interval $[0, t_i]$ i 0 euros si s'emet després. Cap dels n anuncis es pot mostrar més d'una vegada. CinemaVis vol projectar la selecció d'anuncis que li proporcionï màxim benefici. Dissenyeu un algorisme, el més eficient que podeu, per a resoldre aquest problema.

- 2.15. **(Agenda)** A la vostra agenda teniu una llista L de totes les tasques que heu de completar en el dia de avui. Per a cada tasca $i \in L$ s'especifica la durada $d_i \in \mathbb{N}$ que indica el temps necessari per a completar-la i un factor de penalització $p_i \in \mathbb{Z}^+$ que n'agreuja el retard. Heu de determinar en quin ordre realitzar totes les tasques per obtenir el resultat que menys penalització total acumuli.

Tingueu en compte que:

- en un instant de temps només podeu realitzar una única tasca,
- una vegada comenceu a fer una tasca, heu de continuar-la fins a finalitzar-la, i
- s'han de completar totes les tasques.

El criteri d'optimització és la penalització total que s'acumula. La penalització real associada a una tasca $i \in L$ és el temps de finalització t_i de la seva realització, multiplicat per la seva penalització p_i . El temps de finalització t_i es correspon al temps transcorregut des de l'inici de la jornada laboral (és a dir, des de l'instant de temps 0) fins al moment en què s'ha finalitzat la tasca.¹

Considereu l'algorisme voraç que programa les tasques en ordre decreixent de factor de penalització p_i . Determineu si aquest algoritme resol el problema. En cas que no ho faci, proporcioneu un algorisme (tant eficient com pugueu) per resoldre'l.

- 2.16. **(WebSocial)** A l'empresa *WebSocial* li han donat un premi molt important per la seva contribució tecnològica a l'estudi de les relacions interpersonals a xarxes socials. Per celebrar-ho han decidit organitzar una gran festa amb els seus clients en què volen fer gala dels bons resultats que obtenen.

Els CEOs de WebSocial han determinat un conjunt d' n tasques que s'han de portar a terme per a la correcta organització de la festa, i disposen d'un conjunt S d' n treballadors per encarregar-se'n. Totes les tasques requereixen un encarregat i totes menys una requereixen que una altra persona del grup de treballadors seleccionats actuï com a supervisor. Un treballador pot supervisar cap, una o més d'una tasca.

Fent ús de la seva pròpia tecnologia l'equip directiu de WebSocial ha mesurat, per a cada parell de treballadors (a, b) , un coeficient que indica la insatisfacció derivada del fet que un d'ells hagi de supervisar una tasca encarregada a l'altre. També ha determinat un subconjunt de treballadors $I \subseteq S$

¹Observeu que, segons les restriccions del problema, la realització d'una tasca $i \in L$ amb temps de finalització t_i haurà començat a l'instant $t_i - d_i$.

que no poden ser supervisors de cap tasca (tot i ells sí que poden ser supervisats), i també un conjunt de treballadors $J \subseteq S$, $I \cap J = \emptyset$, que podrien fer-se càrrec de tasques sense necessitat de cap supervisió.

També han fixat un llindar màxim d'incompatibilitat u .

- (a) Doneu un algorisme amb cost polinòmic per determinar si és possible trobar una estructura jeràrquica compatible amb les restriccions de WebSocial de manera que, entre tot parell de supervisor i supervisat, el nivell d'insatisfacció sigui com a màxim u .
- (b) Assumint que hi ha una estructura jeràrquica compatible amb les restriccions de l'apartat (a), proporcioneu un algorisme eficient per obtenir una estructura jeràrquica en la qual es minimitzi la suma dels coeficients d'insatisfacció entre supervisors i supervisats.

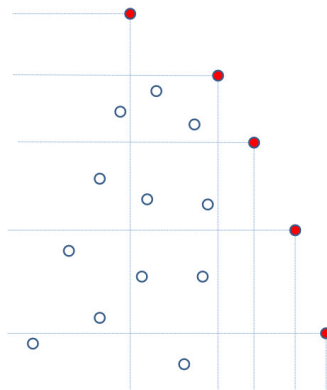
2.17. (**Fusionant seqüències**) Ja sabeu que fer la fusió ordenada de dues seqüències ordenades d' m i n elements, respectivament, comporta fer $m + n$ moviments de dades (penseu, per exemple, en un *merge* durant l'ordenació amb *mergesort* d'un vector). Però si hem de fer la fusió d' N seqüències, dos a dos, l'ordre en què es facin les fusions és rellevant. Imagineu que tenim tres seqüències A , B i C amb 30, 50 i 10 elements, respectivament. Si fusionem A amb B i després el resultat el fusionem amb C , farem $30 + 50 = 80$ moviments per a la primera fusió i $80 + 10 = 90$ per a la segona, amb un total de 170 moviments. En canvi, si fusionem primer A i C i el resultat el fusionem amb B farem un total de 130 moviments.

Dissenyau un algorisme golafre (*greedy*) per fer les fusions i obtenir la seqüència final ordenada amb mínim nombre total de moviments. Justifiqueu la seva correctesa i calculeu-ne el cost temporal del vostre algorisme en funció del nombre de seqüències N .

2.18. (**Òptims de Pareto**)

Considereu un conjunt $P = \{(x_1, y_1), \dots, (x_n, y_n)\}$ d' n punts en el pla Euclidià (2-dimensional). Diem que un punt (x_i, y_i) *domina* un altre punt (x_j, y_j) si és més gran en totes dues coordenades, és a dir, si $x_i > x_j$ i $y_i > y_j$. Diem que un punt és *maximal* si no és dominat per cap altre.

Per exemple, al conjunt de punts mostrat en aquesta figura, els punts sòlids són els punts maximals, mentre que els punts buits són dominats. Les línies discontinües emmarquen quins punts són dominats per cadascun dels punts maximals.



Donat un conjunt P d' n punts, dissenyau un algorisme eficient per a trobar tots els punts maximals de P (els anomenats *Òptims de Pareto*).

2.19. (**Streaming**) Tenim n paquets de vídeo que s'han d'enviar seqüencialment per una única línia de comunicació (això s'anomena *streaming*). El paquet i -èsim té una grandària de b_i bits i triga t_i segons

a travessar, on t_i i b_i són enters positius (no es poden enviar dos paquets al mateix temps). Podeu assumir que la velocitat de transmissió d'un paquet es uniforme.

Hem de decidir una planificació de l'ordre en què hem d'enviar els paquets de manera que, un cop tenim un ordre, no pot haver-hi retard entre la fi d'un paquet i el començament del següent. Assumirem que comencem a l'instant 0 i finalitzem al $\sum_{i=1}^n t_i$. A més, el proveïdor de la connexió vol utilitzar una amplada de banda no massa gran, per tant s'afegeix la restricció següent: Per a cada enter $t > 0$, el nombre total de bits que enviem de temps 0 a t ha de ser $\leq rt$, per a una $r > 0$ fixada (noteu que aquesta restricció no diu res per a períodes de temps que no comencen a l'instant 0). Direm que una planificació és *vàlida* si satisfà la restricció prèvia.

El problema a resoldre és el següent: donat un conjunt de n paquets, cadascun especificat per la seva grandària b_i i la durada de la seva transmissió t_i , i donat el valor del paràmetre r , hem de determinar si existeix una planificació vàlida dels n paquets. Per exemple, si $n = 3$ amb $(b_1, t_1) = (2000, 1)$, $(b_2, t_2) = (6000, 2)$ i $(b_3, t_3) = (2000, 1)$ amb $r = 5000$, aleshores la planificació 1, 2, 3 és vàlida, donat que compleix la restricció.

Per a resoldre aquest problema, heu de resoldre els apartats següents:

- (a) Demostreu o doneu un contraexemple a l'enunciat de: és veritat que existeix una planificació vàlida si, i únicament si, per a cada paquet i tenim $b_i \leq rt_i$.
- (b) Doneu un algorisme que per a una entrada de n paquets (cadascun especificat per (b_i, t_i)) i el paràmetre r , determini si existeix una planificació vàlida. El vostre algorisme hauria de tenir complexitat polinòmica en n .

2.20. **(CatCar)** Un grup de persones que necessiten esporàdicament un cotxe han creat CatCar, una cooperativa per gestionar l'ús compartit dels seus cotxes. La companya té una web, quan una membre de CatCar necessita un cotxe, la nit prèvia, escriu a la pàgina web de la CatCar, l'hora i lloc on vol recollir el cotxe i l'hora i lloc on el deixarà. CatCar ha de preveure tenir suficients cotxes disponibles a cada lloc per a satisfer les demandes del dia, però, per altra banda, també és important tenir un excés de cotxes immobilitzats a un lloc. Per tant, CatCar vol un algorisme eficient que minimitzi el nombre total de cotxes que la companya ha de deixar a cada lloc.

Formalment, el problema és el següent: Com a entrada tenim m llocs (1 fins a m) i n tuples (l, t, l', t') , a on l, t és el lloc i temps de recollida i l', t' és el lloc i tems de retornar el cotxe. Quan una persona s'emporta un cotxe d'un lloc determinat, el nombre de cotxes al lloc disminueix en 1, i quan el torna, incrementa en 1. Doneu un algorisme que calculi el nombre de cotxes que CatCar ha de deixar a cada un dels m llocs, amb les restriccions que cada lloc sempre ha de tenir un nombre no negatiu de cotxes i que el nombre de cotxes immobilitzats ha de ser mínim.

2.21. **(MST?)** Argumenteu si el següent algorisme per a trobar el MST d'un graf donat $G = (V, E)$, que és connex, no dirigit i amb pesos $w : E \rightarrow \mathbb{N}^+$, és o no és correcte, i si és correcte doneu la seva complexitat:

- (a) Particioneu V en dos subconjunts S_1 i S_2 (i.e. $V = S_1 \cup S_2$ i $S_1 \cap S_2 = \emptyset$ tal que cadascun dels subgrafs resultants G_{S_1} i G_{S_2} son connexes.
- (b) Recursivament trobeu un MST T_{S_1} per a G_{S_1} i un MST T_{S_2} per a G_{S_2} .
- (c) Com G és connex hi hauran arestes entre els vèrtexs de T_{S_1} i de T_{S_2} , escolliu l'atesta amb menys pes per a formar un MST de G .

2.22. **(About Huffman)** Per cadascuna de les afirmacions següents indiqueu-ho si és o no certa.

- (a) En un codi de Huffman, si tots els caràcters tenen freqüència $< 1/3$, aleshores **NO** existeix cap caràcter que es pugui codificar amb longitud 1.
- (b) En un codi de Huffman, si tots els caràcters tenen freqüència $< 2/5$, aleshores no hi han caràcters que es pugui codificar amb longitud 1.

- (c) Si a un codi Huffman tenim una fulla a profunditat 1 i una altra fulla a profunditat 3, això implica que ha d'haver una fulla a profunditat 2.

2.23. (Shell decomposition)

Amb l'aparició de les xarxes socials modernes han sorgit nous problemes i elements a estudiar que són de gran interès algorímic. Un d'ells, per exemple, és l'anàlisi de la importància dels actors (usuaris) de la xarxa en relació a un procés de transmissió d'influència que estigui passant en un moment donat. Per a estudiar això s'utilitza una descomposició particular de la xarxa que s'anomena *shell decomposition* (descomposició de closca); per tal d'entendre en què consisteix aquesta descomposició, necessitem conèixer els conceptes de *k-core* i *k-shell*.

Si tenim una xarxa social G (graf no dirigit) amb n actors (vèrtexs), el *k-core* és el subgraf induït de mida màxima en què tots els vèrtexs tenen com a mínim grau k , amb $0 \leq k \leq n$. La *k-shell* és el conjunt d'actors que pertanyen al *k-core* però no pertanyen al $(k+1)$ -core.² Aleshores, la *shell decomposition* de G és la partició de vèrtexs definida per les *k-shells* de G que no són buides.

Donada una xarxa G , dissenyeu un algorisme amb cost lineal que proporcioni la shell decomposition de G . El vostre algorisme ha d'indicar també, per a cada shell a la descomposició, el corresponent valor de k .

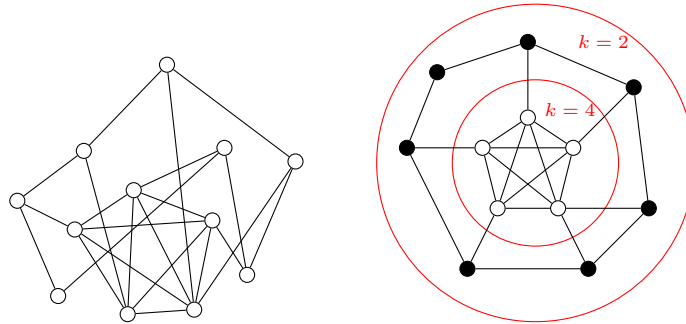


Figura 1: Un graf i una il·lustració de la seva *shell decomposition*

Exemple: Per al graf de la Figura 1 (esquerra), el 4-core és el subgraf induït per els vèrtexs dins del cercle interior que es destaquen a la figura dreta (nodes buits ○), i el 2-core és tot el graf. Per a la resta de *k*-cores: noteu que el 3-core és igual al 4-core, que el 0-core i l'1-core són iguals al 2-core, i que el *k*-core és buit $k \geq 5$.

Per tant, la *shell decomposition* té dues shells no buides: la 4-shell, composta pels vèrtexs del 4-core (nodes buits ○), i la 2-shell, composta pels vèrtexs del 2-core que no són al 3-core (nodes plens ●).

2.24. (Reduint CO_2)

Els nostres polítics han decidit contribuir a la reducció d'emissions de CO_2 compartint el seus vehicles oficials. En assistir a la reunió anual d'alcaldes, un vehicle oficial pot donar servei entre les ciutats A i B, portant a la ciutat B tots els alcaldes que es trobin a A (inclòs l'alcalde d'A). Per a cada possible trajecte (A, B) entre dues ciutats de la província s'ha estimat un valor $c(A, B)$ corresponent a les tones de CO_2 que emet el vehicle oficial en fer aquest trajecte.

Dissenyau un algorisme que, atesa una descripció dels possibles trajectes entre N ciutats, les corresponents estimacions d'emissions de CO_2 i la ciutat on es durà a terme la propera reunió, determini els trajectes que han de ser operats de manera que es minimitzi el total de CO_2 emès i tots els alcaldes puguin arribar a la reunió.

²En alguns grafs pot passar que el *k*-core o la *k*-shell siguin buits per a algunes k .

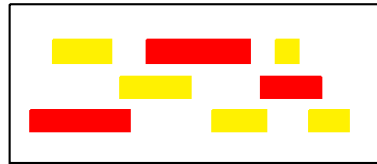
Justifiqueu-ne la correctesa i l'anàlisi del cost en funció del nombre N de ciutats.

Observacions:

- Totes les ciutats tenen alcalde i tots han d'anar a la reunió.
- Inicialment cada alcalde es troba a la seva ciutat.
- El valor $c(A, B)$ és independent del nombre de persones que viatgen al vehicle i del sentit en què es faci el trajecte (és a dir, $c(A, B) = c(B, A), \forall A, B$).
- Els vehicles oficials són molt grans; podeu suposar que hi caben tantes persones com necessiteu.

2.25. **(Nice Buildings)** L'empresa Nice Buildings està especialitzada en pintar façanes de grans edificis que segueixen estrictes criteris d'impacte cromàtic i econòmic. Avui han rebut un encàrrec d'un arquitecte que vol un determinat to de pintura per a les parets i un curiós efecte cromàtic a les finestres de l'edifici que està construint. A les especificacions del projecte s'indica que les finestres són rectangulars, que totes les finestres d'una mateixa planta tenen la mateixa alçada i línia de base, i que cada finestra s'ha de pintar d'un color. A més, a les finestres que estan sobre la mateixa vertical de l'edifici (encara que sigui parcialment) no s'hi pot repetir cap color.

El problema que es troba Nice Buildings és que les finestres no segueixen cap patró regular: tot i que les finestres sí que s'arranglen per pisos, dins d'aquests poden estar col·locades a qualsevol posició i tenir qualsevol amplada. A més, per criteris estètics i econòmics, l'empresa ha de fer servir el *mínim nombre de colors possibles* per pintar les finestres. A la figura podeu veure un exemple de façana d'un edifici en què s'han pogut complir les especificacions de l'arquitecte fent servir només dos colors.



De cada finestra i sabem la planta on es troba (p_i), i les posicions d'inici (x_e) i de final (x_d) de la finestra, mesurades per la distància a la paret de l'esquerra de la façana de l'edifici.

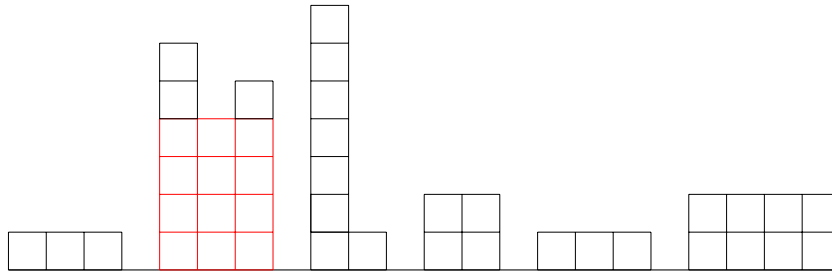
Dissenyeu un algorisme eficient que, donada una llista $\{(p_1, x_{e_1}, x_{d_1}), \dots, (p_n, x_{e_n}, x_{d_n})\}$ que descriu les n finestres de l'edifici, determini una forma de pintar-les respectant les restriccions de l'arquitecte i fent servir el mínim nombre de colors possibles.

2.26. **(Panel)** Volem ajudar al professor Valiente amb el disseny de la forma d'un panel nou per l'entrada del departament de Ciències de la Computació. El professor us dona n columnes, totes de la mateixa amplada, però amb alçades diferents: $h_1, h_2, \dots, h_n$. Les columnes poden reordenar-se a sobre d'una línia base per a definir diferents formes.

El director de CS vol penjar al panel un cartell rectangular.

Si a algun panel hi ha j columnes consecutives amb alçada k o més, podem penjar un rectangle de mides $j \times k$.

A l'exemple de sota tenim un panel amb 3 columnes consecutives amb alçada 4 o més, llavors podem penjar un rectangle d'àrea 12 a sobre d'aquestes 3 columnes.



- (a) Doneu un algorisme polinòmic per trobar l'àrea més gran per poder penjar un cartell quan les columnes segueixen un ordre inicial donat $h_1, h_2, \dots, h_n$, és a dir, quan les columnes no es poden moure.
- (b) Doneu un algorisme polinòmic (i eficient) per reordenar les columnes de manera que l'ordenació proporcioni el panel que maximitzi l'àrea per penjar el cartell.